The Great Labyrinth

1. Project Overview

A 2D loop dungeon game (roguelite) where the player picks a profession (e.g., Warrior, Mage, Rogue), fights procedurally-generated floors, can buy items/skills in a shop, and collects random drops. A *run* progresses floor-by-floor; the nominal end condition is clearing **20 floors** (configurable).

2. Project Review

Reference project: Pixel Dungeon

Pixel Dungeon is a compact, open-source roguelike with procedural floors, item drops, and classes. It is a good baseline for core roguelike mechanics.

Loop-to-20floors game mode — a configurable long-run mode with milestones and per-floor rewards, designed for data collection at scale.

Shop + economy telemetry — detailed tracking of money spent per floor and how purchases affect success metrics.

3. Programming Development

3.1 Game Concept

Goal: Reach floor 20 (or stop earlier if player quits/dies); clear floors to collect money/items. Each full clear yields final rewards.

Player flow: Choose class → Enter dungeon → Fight floors → Collect drops →Optionally visit shop → Repeat until floor 20 or run ends.

Key features / selling points

- Multiple professions with distinct playstyles.
- Persistent per-run progression (money, unlocked cosmetics/skills).
- Randomized drops + shop choices that create meaningful tradeoffs.
- Built-in data logging for every floor (for research/analysis).
- Lightweight controls and 2D pixel-art aesthetic (fast playable sessions).

Endgame & replayability: run statistics, leaderboards (optional), unlocks based on aggregated analytics.

3.2 Object-Oriented Programming Implementation

1.Player

- Role: Represent the human-controlled character and state.
- Attributes: name, class_name, hp, max_hp, ATK, money, inventory (list of Item/Weapon), skills, position, level, mp.
- Key methods: attack(target), use_skill(skill_id, target), take_damage(amount), buy(item), heal(amount), save_state(), move().

2. Entity (base class for enemies & NPCs)

- Role: Generic in-game entity (enemy, neutral NPC).
- Attributes: name, hp, ATK, type, position, loot_table.
- Key methods: is_alive(), take_damage(amount), act(player_state).

3. Weapon (and Item)

- Role: Data containers for equipment/consumables that change player stats or provide effects.
- Attributes: name, price, attack_mod, rarity, type (consumable/equipment), effect.
- Key methods: apply(target), consume(target).

4. Shop

- Role: Handle purchases and restocking between floors or on certain floors.
- Attributes: inventory (list of Items), prices, restock_timer.
- Key methods: show_items(), buy(player, item_id), restock().

5. Dungeon

- Role: Manage floors, spawn enemies, handle floor flow and difficulty scaling.
- Attributes: `current_floor`, `max_floors`, `stats_collector`, `seed`.
- Key methods: `start_run(player)`, `start_floor()`, `end_floor(summary)`, `_spawn_enemies()`.

6. StatsCollector

- **Role:**
  Responsible for collecting, storing, and exporting gameplay data for analysis.
- **Attributes:**
  `combat_log (list of dict) – stores per-encounter data records`

  `shop_log (list of dict) – stores per-shop-visit data records`

  `current_run_id (int) – identifier for each game run`

  `timestamp (datetime) – records time of each event`
- **Key methods:**
  `record_encounter(enemies_killed, damage_taken, damage_dealt, potions_used, time_taken) – saves one combat record`

  `record_shop_visit(money_spent) – saves one shop record`

  `save_to_csv() – exports collected data to CSV files (combat_log.csv, shop_log.csv)`

  `reset_run() – clears data for a new run`

  `get_summary() – returns aggregated statistics for the current run`

-

3.3 Algorithms Involved

Procedural floor generation: seeded RNG to place enemies/loot/rooms (simple cellular automata or random room placement).

Enemy pathfinding: A* or simpler grid-based BFS for short-range navigation.

Enemy: Finite state machine (idle → chase → attack → flee) and simple rule-based logic for decision-making.

Loot drop selection: weighted random sampling from a drop table (alias method or cumulative probability).

Economy / balancing algorithms: simple sorting and threshold checks to decide shop restock; price tiers sorted by rarity.

Event-driven mechanics: event queue to drive combat, item use, and UI events.

4. Statistical Data (Prop Stats)

4.1 Data Features

enemies_killed_in_encounter — integer: number of enemies defeated in one encounter

damage_taken_in_encounter — integer: total damage received in one encounter

damage_dealt_in_encounter — integer: total damage dealt in one encounter

potions_used_in_encounter — integer: number of healing or support items used in one encounter

Money_spent_in_shop — integer: amount of money spent during one shop visit.

4.2 Data Recording Method

Primary storage: CSV files (human-readable, easy for graders)
Implementation: `StatsCollector` class appends a row after every floor clear and at run end. CSV schema ensures consistent fields and timestamps.

4.3 Data Analysis Report

1. Histogram – enemies_killed_in_encounter
A histogram will show the distribution of damage_dealt_in_encounter. This helps identify whether most encounters are low-damage or high-damage situations.

2. Bar Chart – Average Money Spent per Shop Visit
A bar chart will display the average money_spent_in_shop across different shop visits or floors. This helps analyze player spending behavior and resource management.

3. Box Plot – Damage Taken per Encounter
A box plot will visualize damage_taken_in_encounter to show the spread of values and detect outliers. This helps identify unusually difficult encounters.

4. Scatter Plot – Damage Dealt vs Damage Taken
A scatter plot will compare damage_dealt_in_encounter with damage_taken_in_encounter. This helps evaluate player performance (e.g., efficient vs risky combat).

5. Line Chart – Potions Used Over Time
A line chart will show potions_used_in_encounter across encounters. This helps observe how resource usage changes as the run progresses.

Feature Name	Why is it useful? / What is it used for?	How will you obtain 100 values?	Variable & Source class	Display Method
enemies_killed_in_encounter	Measures combat performance and encounter difficulty	Recorded once per encounter; multiple encounters per run will easily generate over 100 records	enemies_killed (StatsCollector)	Histogram
damage_taken_in_encounter	Shows how much damage the player receives and helps evaluate difficulty and survivability	Recorded once per encounter	damage_taken (StatsCollector)	Box plot
potions_used_in_encounter	Tracks resource usage and player strategy during combat	Recorded once per encounter	potions_used (StatsCollector)	Line chart
money_spent_in_shop	Analyzes player spending behavior and decision-making in the shop	Each shop visit is recorded as one entry; multiple visits per run ensure sufficient data	money_spent (Shop / StatsCollector)	Bar chart
Damage_taken_in_encounter vs Damage_Dealt_in_encounter	Measures combat performance	Recorded once per encounter	damage_taken /damage_dealt (StatsCollector)	Scatter Plot

Graph	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis:
Graph1	enemies_killed_in_encounter	Measures combat performance and encounter difficulty	Histogram	enemies_killed_in_encounter	Frequency
Graph2	money_spent_in_shop	To analyze player spending behavior across shop visits	Bar Chart	Shop visit index	Average money_spent_in_shop
Graph3	damage_dealt_in_encounter vs damage_taken_in_encounter	Measures combat performance	Scatter Plot	damage_dealt_in_encounter	damage_taken_in_encounter
Graph4	damage_taken_in_encounter	Measures evade performance	Box plot	damage_taken_in_encounter	Frequency
Graph5	potions_used_in_encounter	To observe how resource usage changes over time during gameplay	Line Chart	Encounter number	potions_used_in_encounter

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Implement player class and basic move
3	10	Implement Map and floor generation
4	11	Implement entity(NPCs+enemies)
5	12	Implement dungeon progression, shop,system, and data logging
6	13	Testing, debugging, balancing the game
7	14	Submission week (Draft)

6. Document version

Version: 2.0
Date: 30 March 2026
Editor: krittin kongsiang